Convection-Diffusion Equation: A Theoretically Certified Framework for Neural Networks

Tangjun Wang, Chenglong Bao, and Zuoqiang Shi

***Abstract*—Differential equations have demonstrated intrinsic connections to network structures, linking discrete network layers through continuous equations. Most existing approaches focus on the interaction between ordinary differential equations (ODEs) and feature transformations, primarily working on input signals. In this paper, we study the partial differential equation (PDE) model of neural networks, viewing the neural network as a functional operating on a base model provided by the last layer of the classifier. Inspired by scale-space theory, we theoretically prove that this mapping can be formulated by a convection-diffusion equation, under interpretable and intuitive assumptions from both neural network and PDE perspectives. This theoretically certified framework covers various existing network structures and training techniques, offering a mathematical foundation and new insights into neural networks. Moreover, based on the convection-diffusion equation model, we design a new network structure that incorporates a diffusion mechanism into the network architecture from a PDE perspective. Extensive experiments on benchmark datasets and real-world applications confirm the effectiveness of the proposed model.**

***Index Terms*—Partial differential equations, neural networks, convection-diffusion equation, scale-space theory.**

I. Introduction

NEURAL networks (NNs) have achieved great success in many tasks, such as image classification [1], speech recognition [2], video analysis [3], and action recognition [4]. Among the existing networks, residual networks (ResNets) are important architectures that enable the training of ultra-deep NNs and have the ability to avoid gradient vanishing [5], [6]. Moreover, the idea of ResNets has motivated the development of many other NNs, including WideResNet [7], ResNeXt [8], and DenseNet [9].

Received 12 June 2024; revised 23 December 2024; accepted 5 February 2025. Date of publication 10 February 2025; date of current version 3 April 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 92370125 and Grant 12271291 and in part by the National Key R&D Program of China under Grant 2021YFA1001300. Recommended for acceptance by H. Su. *(Corresponding authors: Chenglong Bao; Zuoqiang Shi.)*

Tangjun Wang is with the Department of Mathematical Sciences, Tsinghua University, Beijing 100084, China.

Chenglong Bao is with the Yau Mathematical Sciences Center, Tsinghua University, Beijing 100084, China, and also with the Yanqi Lake Beijing Institute of Mathematical Sciences and Applications, Beijing 101408, China (e-mail: clbao@mail.tsinghua.edu.cn).

Zuoqiang Shi is with the Yau Mathematical Sciences Center, Tsinghua University, Beijing 100084, China, and also with the Yanqi Lake Beijing Institute of Mathematical Sciences and Applications, Beijing 101408, China (e-mail: zqshi@tsinghua.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2025.3540310, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2025.3540310

In recent years, understanding ResNets from a dynamical perspective has become a promising approach [10], [11], [12]. Specifically, assuming $\boldsymbol{x}_0 \in \mathbb{R}^d$ as the input of a ResNet and defining $\mathcal{F}$ as the mapping, the l-th residual block can be realized by

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + \mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) \tag{1}$$

where $\boldsymbol{x}_l$ and $\boldsymbol{x}_{l+1}$ are the input and output of the residual mapping, and $\boldsymbol{w}_l$ is the parameter of the l-th block that will be learned by minimizing the training loss. Let $\boldsymbol{x}_L$ be the output of a ResNet with L blocks, then the classification score is determined by $\boldsymbol{y} = \text{softmax}(\boldsymbol{w} \cdot \boldsymbol{x}_L)$, where $\boldsymbol{w}$ is a learnable weight of the final linear classifier.

For any $T > 0$, by introducing a temporal partition $\Delta t = T/L$, the residual block represented by (1) can be viewed as the explicit Euler discretization with time step Δt for the following ordinary differential equation (ODE):

$$\frac{\mathrm{d}\boldsymbol{x}(t)}{\mathrm{d}t} = v(\boldsymbol{x}(t), t), \quad \boldsymbol{x}(0) = \boldsymbol{x}_0, \quad t \in [0, T], \tag{2}$$

where $v(\boldsymbol{x}(t), t)$ is a velocity field such that $v(\boldsymbol{x}(t), t) = \mathcal{F}(\boldsymbol{x}(t), \boldsymbol{w}(t))/\Delta t$. This interpretation of ResNets provides a new perspective for viewing NNs and has inspired the development of many networks. Some approaches involve applying different numerical methods to construct diverse discrete layers [13], [14], [15], while others explore the continuous-depth model [12], [16].

Furthermore, the connection between ODEs and partial differential equations (PDEs) through the well-known characteristics method has motivated the analysis of ResNets from a PDE perspective. This includes theoretical analysis [17], novel training algorithms [18], and improvements in adversarial robustness [19] for NNs. Specifically, define $u : \mathbb{R}^d \times \mathbb{R} \to \mathbb{R}$ as the solution of the convection equation:

$$\frac{\partial u}{\partial t}(\boldsymbol{x}, t) = -v(\boldsymbol{x}, t) \nabla u(\boldsymbol{x}, t), \quad (\boldsymbol{x}, t) \in \mathbb{R}^d \times [0, T]. \tag{3}$$

Then, along the curve defined by (2),

$$\begin{aligned} \frac{\mathrm{d}u(\boldsymbol{x}, t)}{\mathrm{d}t} &= \frac{\partial u}{\partial \boldsymbol{x}}(\boldsymbol{x}, t) \frac{\mathrm{d}\boldsymbol{x}(t)}{\mathrm{d}t} + \frac{\partial u}{\partial t}(\boldsymbol{x}, t) \\ &= \nabla u(\boldsymbol{x}, t) v(\boldsymbol{x}, t) + \frac{\partial u}{\partial t}(\boldsymbol{x}, t) = 0 \end{aligned}$$

which means $u(\boldsymbol{x}, t)$ remains constant in time. This technique is called the method of characteristics in PDE theory. Consequently, denote the flow map from $\boldsymbol{x}(0)$ to $\boldsymbol{x}(T)$ along (2) as Φ, which is the continuous form of feature extraction in ResNets.

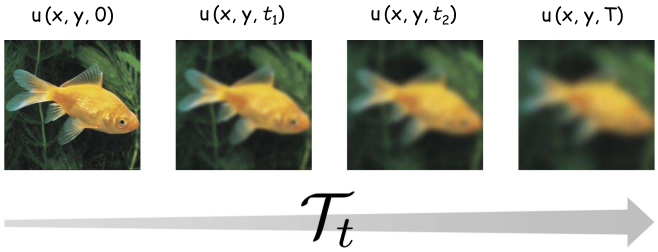

Fig. 1. This work is inspired by scale space theory in image processing, where an image is progressively smoothed over time. The evolution operator $\mathcal{T}_t$, governed by certain PDEs, drives the transformation from the original image to a smoothed image.

If we enforce $u(\boldsymbol{x}, T) = f(\boldsymbol{x}) := \text{softmax}(\boldsymbol{w} \cdot \boldsymbol{x})$ as a linear classifier at $t = T$, then

$$u(\boldsymbol{x}(0), 0) = u(\boldsymbol{x}(T), T) = f(\boldsymbol{x}(T)) = f \circ \Phi(\boldsymbol{x}(0))$$

Thus at $t = 0$, $u(\cdot, 0)$ is the composition of a feature extractor and a linear classifier, which corresponds to a ResNet.

In one word, NN can be viewed as the image $u(\cdot, t)$ of a mapping driven by a certain PDE. In the case of ResNets, this mapping is formulated as a convection equation. A natural question is: *How to bridge NN and PDE in a unified framework?*

In this paper, we try to address the questions from a mathematical perspective. To begin, we formally define the mapping $\mathcal{T}_t$ as follows:

$$\mathcal{T}_t : f = u(\cdot, 0) \mapsto u(\cdot, t), \quad t \in [0, T]$$

This operator converts a base model into a neural network. An illustration of our approach is provided in Fig. 2. Here we choose $u(\boldsymbol{x}, 0) = f(\boldsymbol{x})$ because it is more intuitive to conceptualize NNs as progressing from shallow to deep in the forward direction. The connection between convection equation and ResNets remains consistent, since (3) is reversible in time.

Defining such an evolution operator is inspired by the scale-space theory [20], [21], which is a framework widely used in image processing, computer vision and many fields. A simple example is provided in Fig. 1. It provides formalized theory for manipulating the image at different scales. Scale, in this context, measures the degree of smoothing, or more specifically, the size of neighborhoods of the smoothing kernel. Scale space is the family of smoothed images parameterized by the scale, from the finest image (original image) to the most coarse image. Previous works [22], [23], [24], [25] relate scale-space theory to PDEs, where the smoothing kernels are governed by a set of axioms and satisfy a certain form of PDEs. In this paper, we aim to identify a set of criteria that the operator $\mathcal{T}_t$ should satisfy. By meeting these assumptions, we can derive the form of PDEs, and thus answer the aforementioned questions.

Our framework shares certain similarities with that in scale-space theory while also possessing distinct features. Both frameworks rely on several PDE-type assumptions, such as locality and regularity, because both are built upon PDEs. However, there are notable differences between the two: (1) Scale-space theory typically operates in low-dimensional spaces, such as 2D for images and 3D for movies, whereas NNs can be potentially high-dimensional. (2) NNs possess unique assumptions that are not universal in scale-space theory. On the other hand, certain assumptions from image processing, such as rotation invariance and scale invariance, cannot be directly applied to NNs. (3) The intuition behind similar assumptions may differ between the two frameworks, such as the comparison principle. We argue that our explanation of several assumptions is more natural from a NN viewpoint.

In what follows, we theoretically prove that under reasonable assumptions on $\mathcal{T}_t$, $u(\boldsymbol{x}, t) = \mathcal{T}_t f(\boldsymbol{x})$ is the solution of a second order convection-diffusion PDE,

$$\frac{\partial u(\boldsymbol{x}, t)}{\partial t} = v(\boldsymbol{x}, t) \cdot \nabla u(\boldsymbol{x}, t) + \sum_{i,j} \sigma_{i,j} \frac{\partial^2 u}{\partial x_i \partial x_j}(\boldsymbol{x}, t)$$

We believe that an axiomatic formulation can improve the interpretability of NNs. Establishing the connection between PDE and NN can help us analyze the properties of neural networks and design novel network architectures from a mathematical perspective. Given the wealth of theories in the field of PDE, we are confident that such connection can significantly enhance our comprehension of neural networks. For instance, EnResNet [19] approximates the solution to the convection-diffusion equation by injecting Gaussian noise. This approach aims to enhance NN robustness against adversarial attacks by leveraging insights from PDEs. [26] has developed novel privacy-preserving algorithms inspired by PDEs, without incurring excessive computational overhead or compromising the utility of the models. These algorithms aim to prevent the leakage of specific individual information from the training dataset through the trained neural network. Despite these advancements, a lack of a unified framework exists to provide guidance on the specific selection of PDEs tailored for enhancing NN functionalities. Our theoretical result provides a unified framework which covers various existing network structures such as diffusive graph neural network, and training algorithm designed to improve robustness such as randomized smoothing. The framework also illuminates new thinking for designing networks. Specifically, we propose a new network structure called **CO**nvection d**I**ffusion **N**etworks (COIN), which achieves state-of-the-art or competitive performance on several benchmarks as well as novel tasks.

II. A Theoretically Certified Framework

In this section, we will first propose a theoretically certified framework, supported by several assumptions accompanied by detailed intuition for each assumption. Next, we will describe several examples that can be integrated into our framework. As our motivation is derived from scale-space theory, we will provide a comparison of the meanings implied by the same assumption in two domains. Finally, we will propose a practical algorithm.

A. Assumptions and Theorem

We first show that under several reasonable assumptions, the sequence of operator images $u(\boldsymbol{x}, t) = \mathcal{T}_t f(\boldsymbol{x})$ is the solution of the convection-diffusion equation. Throughout this section, we assume $\mathcal{T}_t$ is well defined on C_b^∞, where C_b^∞ is the space of

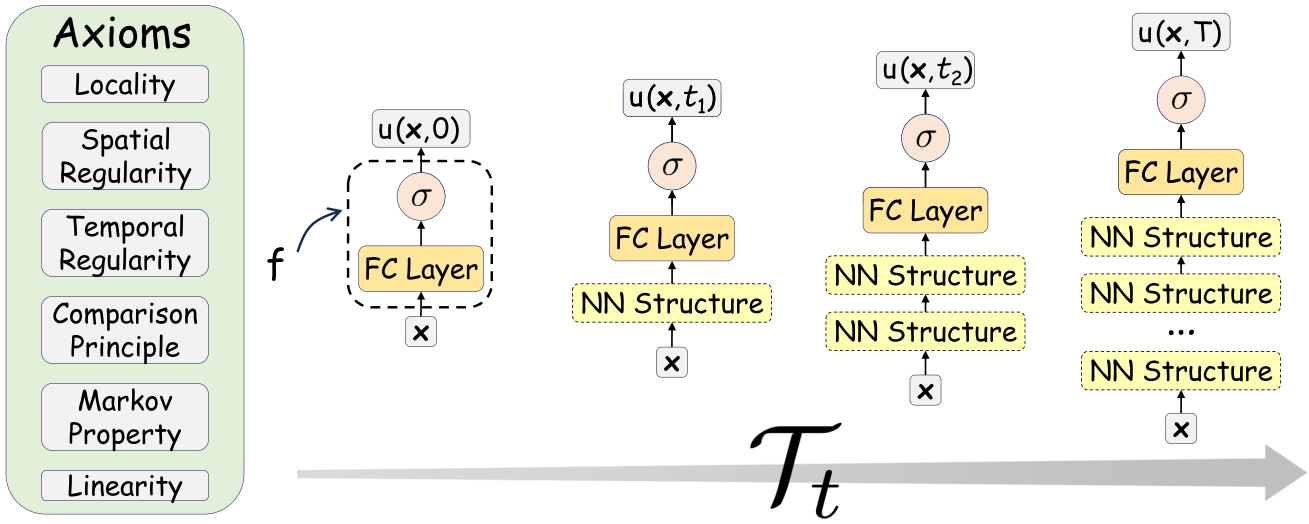

Fig. 2. An illustration of our theoretically certified framework for neural networks. We model the evolution process from shallow to deep neural networks using an operator $\mathcal{T}_t$, which formally signifies the mapping from $f = u(\cdot, 0)$ to $u(\cdot, t)$. By establishing several fundamental axioms, we demonstrate that the operator $\mathcal{T}_t$ is governed by a convection-diffusion equation.

bounded functions which have bounded derivatives at any order, and $\mathcal{T}_t f$ is a bounded continuous function. These assumptions are reasonable, as typical base classifiers f are indeed bounded (between 0 and 1) and have bounded derivatives. The operator image $\mathcal{T}_t f$, which we hope to be a NN, is evidently bounded and continuous.

To get the expression of the operator $\mathcal{T}_t$, we assume it has some fundamental properties, which fall into two categories: NN-type and PDE-type. We will present the assumptions individually and provide a concise explanation of their underlying intuition.

1) NN-Type Assumptions: Neural networks, as one of the most prominent research areas in recent times, exhibit a variety of network architectures, e.g. LeNet [27], AlexNet [28], ResNet [5], Transformer [29]. Despite the apparent differences in network design, we posit that there are shared traits. We distill these properties and formulate them as NN-type assumptions listed below.

Comparison Principle For all $t \geq 0$ and $f, g \in C_b^\infty$, if $f \leq g$, then $\mathcal{T}_t(f) \leq \mathcal{T}_t(g)$.

Suppose we are given two classifiers f and g such that $f(\boldsymbol{x}) \leq g(\boldsymbol{x})$ for all data point $\boldsymbol{x} \in \mathbb{R}^d$. Then $f \circ \Phi(\boldsymbol{x}) \leq g \circ \Phi(\boldsymbol{x})$ if we replace the data points $\boldsymbol{x}$ with the extracted features $\Phi(\boldsymbol{x})$. Recall that for ResNet, $f \circ \Phi = \mathcal{T}_T(f)$, which implies $\mathcal{T}_T(f) \leq \mathcal{T}_T(g)$. Since the order-preserving property holds both at initial time step $t = 0$ and final time step $t = T$, it is reasonable to make the assumption.

Markov Property For all $s, t \geq 0$ and $t + s \leq T$, $\mathcal{T}_{t+s} = \mathcal{T}_t \circ \mathcal{T}_{t+s,t}$, where $\mathcal{T}_{t+s,t}$ denotes the flow from time t to time $t + s$.

The prediction of a deep neural network is computed using forward propagation, i.e. the network uses output of former layer as input of current layer. Thus, for a NN model, it's natural that the output of a NN can be deduced from the output of intermediate l-th layer without any information depending upon the original data point $\boldsymbol{x}$ and output of m-th layer ($m < l$). Regarding the evolution of operator $\mathcal{T}_t$ as stacking layers in the neural network, we should require that $\mathcal{T}_{t+s}$ can be computed from $\mathcal{T}_t$ for any $s \geq 0$, and T_0 is of course the identity.

Linearity For any $f, g \in C_b^\infty$, and real constants β_1, β_2, we have

$$\mathcal{T}_t(\beta_1 f + \beta_2 g) = \beta_1 \mathcal{T}_t(f) + \beta_2 \mathcal{T}_t(g)$$

if C is a constant function, then $\mathcal{T}_t(C) = C$.

Linearity is also an intrinsic property of deep neural networks. Notice that we are not referring to a single NN's output v.s. input linearity, which is obviously wrong because of the activation function. Rather, we are stating that two different NN with the same feature extractor can be merged in to a new NN with a new classifier composed with the shared extractor, i.e.

$$(\beta_1 f + \beta_2 g) \circ \Phi = \beta_1 f \circ \Phi + \beta_2 g \circ \Phi$$

This is linearity at $t = T$, and for $t = 0$ it is trivial. For the latter part, we can hope that a constant base model always produces constant values with evolution.

2) PDE-Type Assumptions: Given our establishment of the relationship between NNs and PDEs in the introduction, it is natural for us to characterize the operator $\mathcal{T}_t$ from the perspective of PDEs. Hence, we introduce the following PDE-type assumptions to ascertain the existence of a differential equation and to demonstrate certian spatial and temporal regularities.

Locality For all fixed $\boldsymbol{x}$, if $f, g \in C_b^\infty$ satisfy $D^\alpha f(\boldsymbol{x}) = D^\alpha g(\boldsymbol{x})$ for all $|\alpha| \geq 0$, where $D^\alpha f$ denotes the α-order derivative of f, then

$$\lim_{t \to 0^+} \frac{(\mathcal{T}_t(f) - \mathcal{T}_t(g))(\boldsymbol{x})}{t} = 0$$

First of all, we need an assumption to ensure the existence of a differential equation. If two classifiers f and g have the same derivatives of any order at some point, then we should assume same evolution at this point when t is small. The concept of

locality in PDEs refer to how the behavior of a solution at a given point depends on the behavior in a neighborhood around that point. In our assumption, this means value of $\mathcal{T}_t(f)$ for t small, at any point $\boldsymbol{x}$, is determined by the behaviour of f near $\boldsymbol{x}$. If we unrigorously define $\partial T_t(f)/\partial t = (\mathcal{T}_t(f) - f)/t$ when $t \to 0^+$ (or infinitesimal generator in our proof), then $\partial T_t(f)/\partial t$ should equal to $\partial T_t(g)/\partial t$. Thus, we give the locality assumption concerning the local character of the operator $\mathcal{T}_t$ for t small.

Spatial Regularity There exists a positive constant C depending on f such that

$$\|\mathcal{T}_t(\tau_{\boldsymbol{h}} f) - \tau_{\boldsymbol{h}}(\mathcal{T}_t f)\|_{L^\infty} \leq Cht$$

for all $f \in C_b^\infty, \boldsymbol{h} \in \mathbb{R}^d, t \geq 0$, where $(\tau_{\boldsymbol{h}} f)(\boldsymbol{x}) = f(\boldsymbol{x} + \boldsymbol{h})$ and $\|\boldsymbol{h}\|_2 = h$.

Regularity is an essential component in PDE theory. Many theorems concerning the existence and uniqueness of solutions to PDEs require regularity as a fundamental property. Thus, when considering PDE-type assumptions on $\mathcal{T}_t$, it is necessary to study its regularity. We separate the regularity requirements into spatial and temporal. Spatial regularity posits that the addition of a perturbation $\boldsymbol{h}$, whether applied to the base model f or the evolved model $T_t f$, should result in minimal differences. This property is essential for predictability in physical systems modeled by PDEs. For example, in heat equation, if we add a small initial temperature change, the change in the resulting temperature distribution will be proportionally small. One can relate it to the well-known translation invariance in image processing, but our assumption is weaker, as we allow small difference rather than require strict equivalence.

Temporal Regularity For all $t, s, t + s \in [0, T]$ and all $f \in C_b^\infty$, there exist a constant $C \geq 0$ depending on f such that

$$\|\mathcal{T}_{t+s,s}(f) - f\|_{L^\infty} \leq Ct$$
$$\|\mathcal{T}_{t+s,s}(f) - \mathcal{T}_t(f)\|_{L^\infty} \leq Cst$$

Temporal regularity requires that in any small time interval, the evolution process will not be rapid, because we want a smooth operator $\mathcal{T}_t$ in time. Taking the heat equation as an illustration again, the temperature at a certain point should not suddenly change over time due to temporal regularity. Conversely, the solution to the heat equation evolves smoothly over time, and gradually converges to a steady state.

Finally, combine all the assumptions on $\mathcal{T}_t$, we can derive the following theorem, emphasizing that the output value of neural network $T_t(f)$ with time evolution satisfies a convection-diffusion equation,

Theorem 1: Under the above assumptions, there exists Lipschitz continuous function $v : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$ and Lipschitz continuous positive function $\sigma : \mathbb{R}^d \times [0, T] \to \mathbb{R}^{d\times d}$ such that for any bounded and uniformly continuous base classifier $f(\boldsymbol{x})$, $u(\boldsymbol{x}, t) = \mathcal{T}_t(f)(\boldsymbol{x})$ is the unique solution of the following convection-diffusion equation:

$$\begin{cases} \frac{\partial u(\boldsymbol{x},t)}{\partial t} = v(\boldsymbol{x}, t) \cdot \nabla u(\boldsymbol{x}, t) + \sum_{i,j} \sigma_{i,j} \frac{\partial^2 u}{\partial x_i \partial x_j}(\boldsymbol{x}, t), \\ u(\boldsymbol{x}, 0) = f(\boldsymbol{x}), \end{cases} \tag{4}$$

where $\boldsymbol{x} \in \mathbb{R}^d, t \in [0, T]$. Here $\sigma_{i,j}$ is the i, j-th element of matrix function $\sigma(\boldsymbol{x}, t)$.

We will provide the proof of Theorem 1 in the appendix.

Remark 1: Why do we want to use an axiomatic framework? Why do we say it can improve the interpretability of NNs? Some may question this, arguing that since the analytic expression of solutions to convection-diffusion equations is intractable, the framework has nothing to do with interpretability. However, it is important to clarify that we are referring to the interpretability of the model itself, rather than the interpretability of the model results. An instructive would be Maxwell's equations, which serve as the foundation of classical electromagnetism. These equations are derived from several fundamental laws in physics, such as Gauss's law and Faraday's law. They help scientists understand the underlying principles governing electricity, magnetism, and light. Although it is generally not possible to solve Maxwell's equations analytically, this does not diminish the interpretability provided by the equations. Similarly, in our framework, the solution to the convection-diffusion model is not the primary concern; rather, it is the assumptions made and the intuition behind those assumptions that truly matter.

B. Examples under framework

Under the convection-diffusion framework, we can give interpretation to several regularization mechanisms that are designed to improve robustness, including Gaussian noise injection [19], [30], ResNet with stochastic dropping out the hidden state of residual block [18], [31] and randomized smoothing [32], [33], [34]. We can also interpret several graph neural networks, including diffusive ones [35], [36], [37], [38] and convective ones [39], [40], as a specific example under our unified framework.

Gaussian noise injection Gaussian noise injection is an effective regularization mechanism for a NN model. For a vanilla ResNet with L residual mapping, the l-th residual mapping with Gaussian noise injected can be written as

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + \mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) + a\mathcal{N}(0, \mathbf{I})$$

where the parameter a is a noise coefficient. Using the same temporal partition as in the introduction, and let $a = \sigma\sqrt{\Delta t}$, this noise injection can be viewed as the approximation of the following continuous dynamic using the Euler-Maruyama method,

$$\mathrm{d}\boldsymbol{x}(t) = v(\boldsymbol{x}(t), t)\mathrm{d}t + \sigma \mathrm{d}\mathbf{B}(t) \tag{5}$$

where $\mathbf{B}(t)$ is a Brownian motion. The output of L-th residual mapping is the state of Itô process (5) at terminal time T. So, an ensemble prediction over multiple networks with shared parameters and random Gaussian noise can be written as a conditional expectation,

$$\hat{y} = \mathbb{E}(\mathrm{softmax}(\boldsymbol{w}\boldsymbol{x}(T))|\boldsymbol{x}(0) = \boldsymbol{x}_0). \tag{6}$$

According to Feynman-Kac formula [41], (6) is known to solve the following convection-diffusion equation

$$\begin{cases} \frac{\partial u(\boldsymbol{x},t)}{\partial t} = v(\boldsymbol{x}, t) \cdot \nabla u(\boldsymbol{x}, t) + \frac{1}{2}\sigma^2 \Delta u(\boldsymbol{x}, t), \quad t \in [0, T] \\ u(\boldsymbol{x}, 0) = \mathrm{softmax}(\boldsymbol{w}\boldsymbol{x}). \end{cases}$$

Dropout of Hidden Units Consider the case that we disable every hidden units independently from a Bernoulli distribution $\mathcal{B}(1,p)$ with $p \in (0,1)$ in each residual mapping

$$\begin{aligned}\boldsymbol{x}_{l+1} &= \boldsymbol{x}_l + \mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) \odot \frac{\boldsymbol{z}_l}{p} \\ &= \boldsymbol{x}_l + \mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) + \mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) \odot \left(\frac{\boldsymbol{z}_l}{p} - \mathbf{I}\right)\end{aligned}$$

where $\boldsymbol{z}_l \sim \mathcal{B}(1,p)$ namely $\mathbb{P}(\boldsymbol{z}_n = 0) = 1-p$, $\mathbb{P}(\boldsymbol{z}_n = 1) = p$ and $\odot$ indicates the Hadamard product. If the number of the ensemble is large enough, according to Central Limit Theorem, we have

$$\mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) \odot \left(\frac{\boldsymbol{z}_l}{p} - \mathbf{I}\right) \approx \mathcal{F}(\boldsymbol{x}_l, \boldsymbol{w}_l) \odot \mathcal{N}\left(0, \frac{1-p}{p}\right)$$

The similar way with Gaussian noise injection, the ensemble prediction $\hat{y}$ can be viewed as the solution $u(\boldsymbol{x}, T)$ of an convection-diffusion equation with diffusion term

$$\frac{1-p}{2p} \sum_i (v^T v)_{i,i} \frac{\partial^2 u}{\partial x_i^2}(\boldsymbol{x}, t)$$

Compared with adding Gaussian noise, which is an isotropic model, dropout of hidden units is an anisotropic model, because the noise introduced by dropout is related to the output of previous layer. In fact, similar to dropout, shake-shake regularization [42], [43] and ResNet with stochastic depth [44] can also be interpreted by our convection-diffusion equation model.

Randomized Smoothing Consider transforming a trained classifier into a new smoothed classifier by adding Gaussian noise to the input during inference. If we denote the trained classifier by $h(\boldsymbol{x})$ and denote the new smoothed classifier by $g(\boldsymbol{x})$. Then $h(\boldsymbol{x})$ and $g(\boldsymbol{x})$ have the following relation:

$$g(\boldsymbol{x}) = \frac{1}{N} \sum_{i=1}^{N} h(\boldsymbol{x} + \boldsymbol{\varepsilon}_i) \approx \mathbb{E}_{\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2 I)}[h(\boldsymbol{x} + \boldsymbol{\varepsilon})]$$

where $\boldsymbol{\varepsilon}_i \sim \mathcal{N}(0, \sigma^2 I)$. According to Feynman-Kac formula, $g(\boldsymbol{x})$ can be viewed as the solution of the following PDE

$$\begin{cases} \frac{\partial u(\boldsymbol{x},t)}{\partial t} = \frac{1}{2}\sigma^2 \Delta u, & t \in [0,1] \\ u(\boldsymbol{x}, 0) = h(\boldsymbol{x}). \end{cases}$$

Especially, when $h(\boldsymbol{x})$ is a ResNet, the smoothed classifier $g(\boldsymbol{x}) = u(\boldsymbol{x}, T+1)$ can be expressed as

$$\begin{cases} \frac{\partial u(\boldsymbol{x},t)}{\partial t} = v(\boldsymbol{x},t) \cdot \nabla u(\boldsymbol{x},t), & t \in [0,T] \\ \frac{\partial u(\boldsymbol{x},t)}{\partial t} = \frac{1}{2}\sigma^2 \Delta u, & t \in [T, T+1] \\ u(\boldsymbol{x}, 0) = \mathrm{softmax}(\boldsymbol{w}\boldsymbol{x}). \end{cases}$$

We extend the terminal time T to $T+1$ to emphasize that randomized smoothing is a post-processing step.

Diffusive Graph Neural Network Some models, e.g. PDE-GCN [35], GRAND [36], DGC [37], introduce diffusion in graph neural network, which corresponds to the diffusion part in our convection-diffusion framework. Diff-ResNet [38] contains both convection and diffusion, but its formulation is derived from ODE perspective. Thus the diffusion is applied on the features $\boldsymbol{x}$, rather than on the value $u(\boldsymbol{x}, t)$.

Convective Graph Neural Network There are some methods, including CGNN [39] and GCDE [40], which models the forward propagation from the perspective of ODE,

$$\frac{\mathrm{d}\boldsymbol{x}(t)}{\mathrm{d}t} = v(\boldsymbol{x}(t), t)$$

As stated in the introduction, it can be viewed as the characteristics of the convective PDE,

$$\frac{\partial u(\boldsymbol{x},t)}{\partial t} = -v(\boldsymbol{x},t) \cdot \nabla u(\boldsymbol{x},t), \quad t \in [0,T]$$

Nonetheless, these methods belong to the ODE category, ignoring the diffusion term during forward propagation.

C. Comparison With Scale-Space Theory

We compare our assumptions with the corresponding assumptions in the scale space theory, providing their common ground and difference below.

Comparison Principle In scale-space theory, especially for the transformation on grey-scale images, the comparison principle means that if one grey-scale image is everywhere brighter than another image, this ordering should be preserved along with the smoothing of the original picture. However, when the goal is to find the edges or depth map in the image, the comparison principle is no longer valid. However, from our NN perspective, comparison principle is natural and always valid, since the ordering between classifiers should be the same, no matter the input is raw data point or extracted feature.

Markov Property Markov Property corresponds to *Recursivity*, or slightly weaker *Causality*, in scale space theory. It is natural that a coarser analysis of the original picture can be inferred from a finer one without any reliance on the original picture itself. It is also natural in NN due to the feed-forward structure of network design.

Linearity Linearity is not a common assumption in the field of scale space theory, as there exists both linear processes [20], [22] and nonlinear processes [23], depending on the filters. However, it is an inherent property of NN, as discussed in the theoretical results. Such linear combination is widely used in practice known as ensemble methods [45]. It is a popular method in machine learning which combines many weak classifiers and add them to obtain a strong classifier.

Locality Locality describes the local characterization of the operator, and is crucial for both scale space theory and ours, as they both use PDE to describe the evolution of the operator. Using the assumptions other than Locality, we may prove the existence of an infinitesimal generator, $(\mathcal{T}_t(f) - f)/t$. Then, the meaning of locality is that if two functions have the same derivatives at some point, then they have the same infinitesimal generator at this point.

Spatial Regularity The most relevant axiom in scale-space theory is translation invariance, which means

$$\mathcal{T}_t(\tau_{\boldsymbol{h}} f) = \tau_{\boldsymbol{h}}(\mathcal{T}_t f)$$

It is stronger than our assumption as it requires absolute equivalence. The concept of translation invariance stems from the idea that there should be no prior knowledge about the specific

location of a feature in an image. In convolutional neural networks [27], translation invariance is achieved by a combination of convolutional layers and pooling layers. For example, a cat is recognized as a cat regardless of whether it appears in the top or bottom half of an image. However, we do not intend to manually design the neural network structure to strictly enforce this level of invariance. Instead, we relax the constraint to allow for some variations.

Spatial regularity may also be beneficial for adversarial robustness. NNs have been shown to be vulnerable to some well-designed input samples, which are called adversarial samples [46], [47]. These adversarial samples are produced by adding carefully hand-crafted perturbations to the inputs of the targeted model. Although these perturbations are imperceptible to human eyes, they can fool NNs to make wrong prediction. In some sense, the existence of these adversarial examples is due to spatial unstability of NNs. We hope the new model $\mathcal{T}_t(f)$ to be spatially stable by adding spatial regularity.

Temporal Regularity Temporal regularity is used in both scale-space theory and in our NN framework, which ensures the existence of PDE. The first inequality states a natural assumption about continuity. When $s = 0$, it reduces to

$$\|\mathcal{T}_t(f) - f\|_{L^\infty} \leq Ct$$

which means that the change should be small when the evolution time is short. It is related to stiffness in the field of numerical solution of differential equations. A stiff equation generally means that there is rapid variation in the solution, and thus we need extremely small steps when numerically solving the equation. Stiffness is not a desirable property of differential equations, and thus we require temporal regularity in the framework. The second inequality is a natural extension for time origin from $t = 0$ to $t \geq 0$.

III. Algorithm: Convection Diffusion Network

To validate the effectiveness of our axiomatized PDE model, we have designed a novel network structure called **CO**nvection d**I**ffusion **N**etworks (COIN), which incorporates diffusion layers after the ResNet architecture.

The proposed algorithm is a straightforward split scheme of (4). We divide the convection-diffusion equation into two parts, namely, the convection part and the diffusion part,

$$\begin{cases} \frac{\partial u(\boldsymbol{x},t)}{\partial t} = v(\boldsymbol{x},t)\cdot\nabla u(\boldsymbol{x},t), & t\in[0,T-1] \\ \frac{\partial u(\boldsymbol{x},t)}{\partial t} = \sigma^2\Delta u(\boldsymbol{x},t), & t\in[T-1,T] \\ u(\boldsymbol{x},0) = f(\boldsymbol{x}) \end{cases}$$

Here we focus on the isotropic models, i.e., the diffusion term is $\sigma^2\Delta u$. The time step $T-1$ serves as a pseudo time step for understanding and does not have a practical impact. As mentioned in the introduction, the forward propagation of ResNets can be viewed as the convection equation. Therefore, we can use a ResNet to simulate the convection part from 0 to $T-1$. In our implementation, we use a shallow two-layer fully connected network with a residual connection to represent the ResNet.

To handle the diffusion part, we model the data samples as nodes on a graph, allowing us to discretize the Laplacian term using the graph Laplacian. In this context, a graph represents a discretization of the domain $\mathbb{R}^n$ into a finite space, where a continuous function vector u is defined. Given a graph $\mathcal{G} = (\mathcal{V},\mathcal{E})$, where $\mathcal{V} = \{\boldsymbol{x}_i\}_{i=1}^N$ denotes the set of N vertices and $\mathcal{E} = \{w_{ij}\}_{i,j=1}^N$ describes the relationship between nodes $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$, we can compute the graph Laplacian as follows:

$$\Delta u(\boldsymbol{x},t) = -Lu(\boldsymbol{x},t) = -\sum_{j=1}^{N} w_{ij}\left(u(\boldsymbol{x}_i,t) - u(\boldsymbol{x}_j,t)\right)$$

The weight w_{ij} can be either given or pre-computed, depending on the specific task. $L = D - W$ is the graph Laplacian matrix, where $D = \mathrm{diag}(d_i)$ is the diagonal matrix with entries $d_i = \sum_{j=1}^N w_{ij}$. It is possible to introduce attention mechanism to dynamically adjust the pairwise weight. By learning the attention parameter, we may model more complex relationship, and we leave as future work.

Then, by applying the forward Euler scheme to discretize the derivative with respect to time t, we obtain the following expression:

$$u_i^{k+1} = u_i^k - \sigma^2\sum_j w_{ij}(u_i^k - u_j^k), \quad k = 0,1,\ldots,K-1 \tag{7}$$

where u_i^k represents the value of u on node $\boldsymbol{x}_i$ at time step t_k. The initial time step t_0 is set to $T-1$, which corresponds to the output of the ResNet, and the final time step $t_K = T$ corresponds to the final output of COIN. (7) is referred to as a diffusion layer. In our implementation, we often stack multiple diffusion layers ($K > 1$) because the value of σ^2 cannot be too large due to stability concerns [38]. Consequently, we use the forward Euler scheme to discretize the diffusion term, allowing us to reach the desired diffusion strength.

To demonstrate that our implementation is derived from the PDE, rather than ODE, perspective, we should point out that diffusion is imposed on the network output value u, rather than intermediate feature $\boldsymbol{x}$. We achieve such idea by incorporating the activation function (such as softmax for multi-class classification or sigmoid for binary classification) in the output layer of ResNet represented by $u(\boldsymbol{x},T-1)$. Consequently, when computing the final loss function, it is unnecessary to add an additional activation function after $u(\boldsymbol{x},T)$. The network structure is illustrated in Fig. 3.

We should acknowledge that from a complexity standpoint, directly applying our COIN model to large-scale datasets, e.g. ImageNet [48], is impractical. The primary computational burden arises from the necessity to construct a graph for the entire dataset, which is unfeasible in terms of both time and space for large datasets. Nevertheless, we are exploring strategies to construct a relatively small graph for each mini-batch during training. Last but not least, we want to emphasize that our COIN model is only one of the many possible approaches of modeling the convection-diffusion PDE. Other methods may include introducing a regularization term that enforces NNs to obey the PDE, similar to PINN [49]. We are looking forward to exploring other paths in the future work.

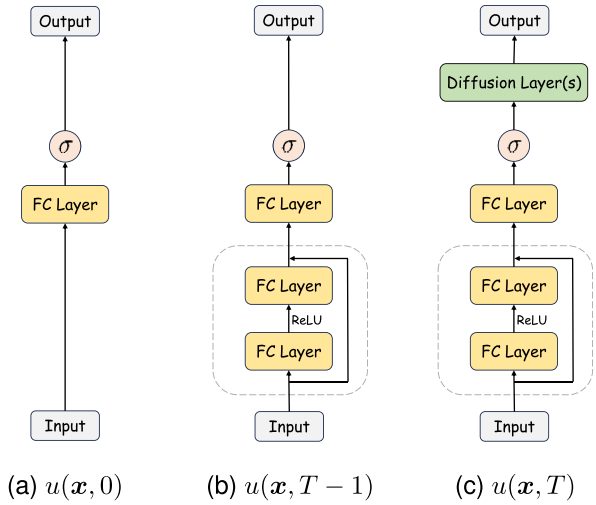

Fig. 3. Illustration for our Convection Diffusion Network at different time step. $u(\boldsymbol{x}, T)$ is the final network structure.

TABLE I
PERFORMANCE COMPARISON ON GRAPH NODE CLASSIFICATION TASKS

	Method	Cora	Citeseer	Pubmed
Classic	MLP	57.4 ± 2.1	59.9 ± 2.2	70.0 ± 2.0
	GCN [53]	81.6 ± 1.1	72.1 ± 1.6	79.0 ± 2.1
	GraphSAGE [54]	79.3 ± 1.4	71.7 ± 1.6	76.1 ± 2.0
	GAT [55]	80.8 ± 1.3	71.6 ± 1.7	78.7 ± 2.1
	SGC [56]	80.5 ± 1.3	73.9 ± 1.4	77.2 ± 2.6
	APPNP [57]	**82.7** ± 1.1	73.3 ± 1.5	80.6 ± 1.8
ODE	CGNN* [39]	82.5 ± 1.0	73.0 ± 1.6	80.5 ± 2.2
	GCDE [40]	80.0 ± 1.5	72.1 ± 1.6	76.0 ± 3.9
Diffusion	GDC [58]	81.6 ± 1.3	72.2 ± 2.6	79.0 ± 2.0
	GraphHeat* [59]	81.4 ± 1.2	73.5 ± 1.5	78.4 ± 2.1
	DGC [37]	81.4 ± 1.2	75.0 ± 1.9	78.2 ± 2.1
	Differmer [60]	82.0 ± 2.3	71.9 ± 1.7	74.8 ± 4.5
	GRAND [36]	82.5 ± 1.4	73.7 ± 1.7	78.8 ± 1.8
	Diff-ResNet [38]	82.1 ± 1.1	74.6 ± 1.8	80.1 ± 2.0
	COIN	82.2 ± 1.2	**75.8** ± 1.3	**81.1** ± 1.9

Reported results are average accuracy ± standard variation from 100*20 experiments. Methods marked with * indicate that their results are averaged across 40 random splits with 10 random initialization each, due to excessive time consumption per task.

IV. EXPERIMENTS

In this section, we show the efficacy of the convection-diffusion framework on graph node classification and few-shot learning benchmarks, as well as novel tasks including COVID-19 case prediction and prostate cancer classification.[1]

A. Graph Node Classification

We have performed tests on our COIN model for semi-supervised node classification problems in graph. We present the results for the well-established citation network benchmarks, namely Cora, Citeseer, and Pubmed [50]. These datasets consist of citation networks where nodes represent publications, edges represent citation links, and features are represented by sparse bag-of-words vectors. The dataset statistics are provided in the appendix. Each node (publication) is categorized into a class. The objective of the graph node classification task is to predict the class of test nodes, given only a limited number of labeled training nodes.

To ensure a reliable comparison, we follow the methodology of Shchur et al. [51] instead of using the fixed Planetoid split [52]. We conduct 100 random train-val-test splits, with each split involving 20 random neural network initialization. We report the average accuracy and standard variation across these splits. For each dataset, we adopt the approach of Shchur et al. [51] and select 20 data points per class for the training set, 30 data points per class for the validation set, and the remaining points for the test set.

We compare our method with graph learning methods from three categories that are closely related to ours: classic methods, ODE-based methods, and methods that also include diffusion. Some methods may belong to more than one category, e.g. GRAND. In this case, we pick its main contribution as the category. We have re-implemented all the aforementioned methods using their official code available online and compared their performance with ours under the same experimental settings. During the testing of these methods, we have used the recommended parameters provided in the paper or the Github repository. To address possible concerns, a comparison between our re-implemented results and reported results is provided in the appendix. For detailed training settings, please refer to the appendix.

The experimental results are presented in Table I. Our COIN outperforms state-of-the-art approaches in terms of accuracy on Citeseer and Pubmed, while achieving comparable results on Cora.

Furthermore, we study the effect of diffusion strength on the network performance. We fix the number of diffusion layers at either $K = 20$ or $K = 40$ and vary the diffusion strength of each layer σ^2. The accuracy is averaged over 10 random train-validation-test splits, with each split involving 10 random neural network initializations, thus potentially yielding slightly different results compared to Table I. We plot the boxplot of 100 experimental results in Fig. 4.

From the results, two findings emerge. First, the performance gradually increases and then decreases with the total diffusion strength $K\sigma^2$, which indicates the existence of an optimal strength. Notably, the network achieves comparable performance within a fairly broad range of diffusion strength. Second, for a given total strength $k\sigma^2$, the layer number K does not have a significant effect. For example, the performance with $K = 20$ and $\sigma^2 = 0.3$ is comparable to that with $K = 40$ and $\sigma^2 = 0.15$.

Regarding the time and space complexity of our algorithm, we provide the average time per training epoch, average convergence time per task, and allocated GPU memory of our COIN, compared to other methods. The experiments are conducted on a single NVIDIA GeForce RTX 3090. It is worth noting that we have used the same early-stopping criterion for all

[1]Code and data for reproducing results in the paper has been deposited in https://github.com/shwangtangjun/COIN.

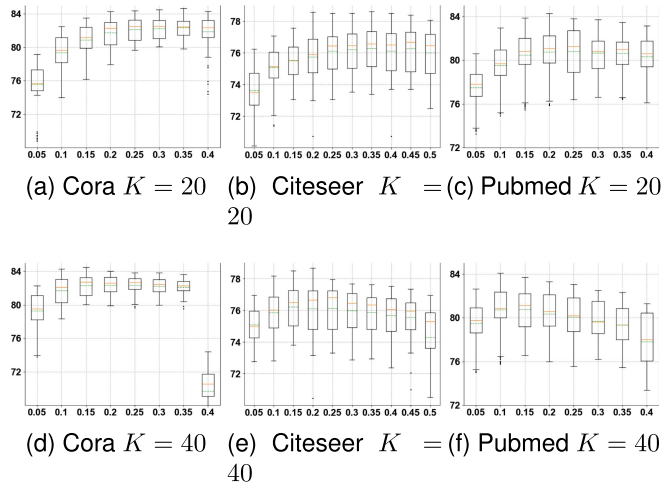

Fig. 4. Accuracy boxplots of COIN with different diffusion strength. x-axis represents σ^2, y-axis represents accuracy(%). The orange solid line represents median. The green dashed line represents mean. The lower and upper hinges correspond to the 1st and 3rd quartiles, the whisker corresponds to the minimum or maximum values no further than 1.5 × inter-quartile range from the hinge. Data beyond the end of the whiskers are outlying points that are plotted individually.

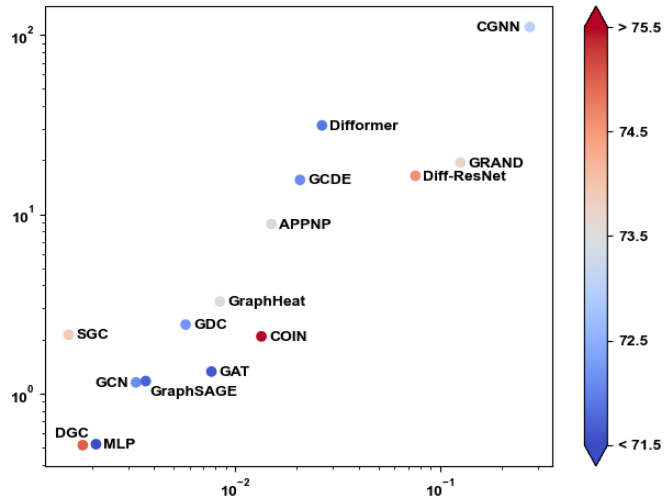

Fig. 5. Time complexity on Citeseer dataset. The x-axis represents average time (seconds) per training epoch, and the y-axis represents average convergence time (seconds) per task, both in log-scale. The color bar measures the average accuracy of each method.

methods, ensuring a fair comparison in terms of average convergence time. Our method incorporates diffusion on the output $u = f(\boldsymbol{x}) \in \mathbb{R}^c$, where c represents the number of classes. As the class number is usually significantly smaller than the feature dimension (e.g., 7 vs. 1433 for Cora dataset), our method exhibits low time and space complexity, even with multiple diffusion layers. As shown in Fig. 5, our method demonstrates significantly reduced per-epoch time and convergence time compared to certain methods in the ODE and Diffusion categories, such as CGNN and GRAND, while achieving superior results. The time complexity results for Cora and Citeseer are provided in the appendix. In Fig. 6, even with multiple diffusion layers ($K = 20$

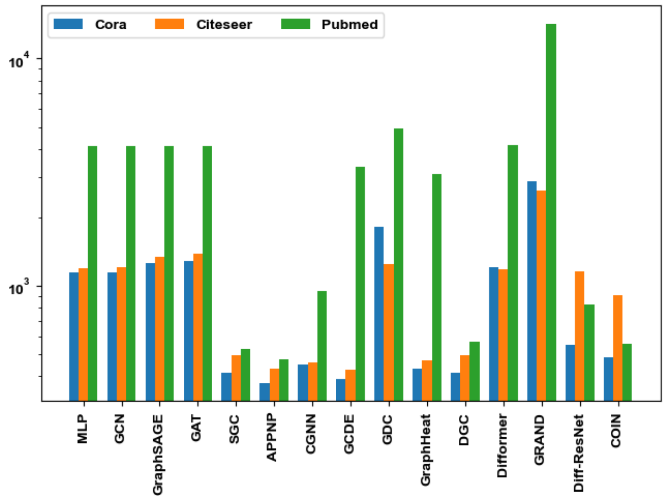

Fig. 6. Space complexity shown in bar plot. The y-axis represents the allocated GPU memory (MB) in log-scale.

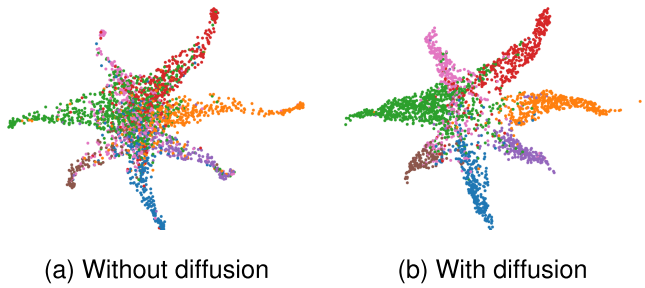

Fig. 7. UMAP visualization for outputs with and without diffusion on Cora dataset.

in graph node classification tasks), COIN does not occupy much memory.

To give an intuitive understanding of the effect of diffusion, we have visualized the outputs both with and without the diffusion mechanism applied to the Cora dataset using UMAP [61], as depicted in Fig. 7. In the absence of diffusion, the outputs of data points belonging to different classes tend to cluster together in the central region. However, by incorporating the diffusion mechanism, this undesired clustering behaviour is notably alleviated.

B. Few-Shot Learning

The effectiveness of deep learning methods is strongly influenced by the availability of a substantial number of training examples. However, collecting such data requires significant labor and is often unfeasible in many domains due to the privacy or safety issues. To alleviate the reliance on training data, there has been a growing interest in few-shot learning methods [62], [63] in recent years. See [64] for a comprehensive review. Formally, the few-shot learning tasks are defined as follows. Given a novel dataset $\mathbb{X}_{\text{novel}} = \mathbb{X}_{\text{s}} \cup \mathbb{X}_{\text{q}}$, where $\mathbb{X}_{\text{s}} = \{(x_i, y_i)\}_{i=1}^{N_1}$ is the support set with label information and $\mathbb{X}_{\text{q}} = \{x_j\}_{j=1}^{N_2}$ is the query set without labels, the goal of few-shot learning is to find

the labels of points in the query set when the size of support set $|N_1|$ is very small. Along with the novel dataset $\mathbb{X}_{\text{novel}}$, there exists a base dataset $\mathbb{X}_{\text{base}}$, where all samples are provided with label information. To prevent information leak, $\mathbb{X}_{\text{base}}$ and $\mathbb{X}_{\text{novel}}$ contain data points from distinct classes. The base dataset can be utilized for various purposes, such as data augmentation in transfer learning, episodic training in meta-learning, or backbone training in embedding learning. Among the different few-shot learning method, we employ embedding learning, which aims to map each sample into a latent space such that similar samples are close while dissimilar samples are far away. The embedding function is parameterized by a deep neural network (backbone) pretrained on $\mathbb{X}_{\text{base}}$. During the few-shot learning tasks on $\mathbb{X}_{\text{novel}}$, the pretrained embedding function remains fixed without any further fine-tuning.

We conduct experiments on three benchmarks for few-shot image classification: *mini*ImageNet, *tiered*ImageNet and CUB. The *mini*ImageNet and *tiered*ImageNet are both subsets of the larger ILSVRC-12 dataset [48], with 100 classes and 608 classes respectively. The *mini*ImageNet contains 100 classes and is split into 64 base classes, 16 validation classes, and 20 novel classes. The *tiered*ImageNet contains 608 classes and is split into 351 base classes, 97 validation classes, and 160 novel classes. CUB-200-2011 [86] is a fine-grained image classification dataset with 200 classes. It is split into 100 base classes, 50 validation classes, and 50 novel classes. The dataset split is standard as in previous papers [77], [78]. All images are resized to 84×84, following [63].

The most common way to build a task is called an N-way-K-shot task [63], where N classes are sampled from $\mathbb{X}_{\text{novel}}$ and only K (e.g., 1 or 5) labeled samples are provided for each class. Following standard evaluation protocol [77], [78], we randomly sample 10000 5-way-1-shot and 5-way-5-shot classification tasks, and report the average accuracy and corresponding 95% confidence interval.

We choose two widely used networks, ResNet-18 [5] and WRN-28-10 [7] as our backbone. The backbone training process is in general similar to that in SimpleShot [77] and Laplacian-Shot [78], but details are slightly different. See the appendix for details. The training process follows a standard pipeline in supervised learning and does not involve any meta-learning or episodic-training strategy. As a result, we obtain an embedding function that maps the original data points to $\mathbb{R}^M$, where $M = 512$ for ResNet-18 and $M = 640$ for WRN-28-10. For fair comparison, we also employ techniques used in [77], [78], including centering and normalization and cross-domain shift, to transform the embedded features. The details of these techniques are explained in the appendix.

During each few-shot task, we train a COIN using M-dimensional features extracted from both the support set and the query set. To apply diffusion, we require a weight matrix that captures the similarity between data points. The weight w_{ij} is computed using a Gaussian kernel based on the Euclidean distance between x_i and x_j.

The results are reported in Table II, in which the results for comparison are collected from [77], [78]. Across all datasets with different backbone architectures, our COIN consistently achieves the highest classification accuracy. Our method outperforms the state-of-the-art by an average margin of more than 1%, showcasing its effectiveness in scenarios with extremely limited training data.

C. COVID-19 Case Prediction With Missing Data

Our axiomatic framework can also be applied to predict the reported case for COVID-19 in scenarios with missing data. During a pandemic, accurate prediction of the infection spread is of utmost importance to enable governments to take timely and proactive measures. Timely and accurate predictions enable governments to make informed decisions, optimize healthcare resource allocation, and implement effective measures to suppress the spread of the virus. Recent studies have addressed the challenge of predicting pandemics using deep learning methods [87], [88], [89], [90]. One promising approach is to use graph neural network [91], [92], [93], [94], where regions are considered as nodes, and the interaction between nodes is captured through human mobility data (i.e., the number of people moving from one place to another within a given period).

We conduct our experiment using the England COVID-19 dataset sourced from the PyTorch Geometric Temporal open-source library [95]. This dataset comprises daily reported COVID-19 cases in 129 regions of England known as NUTS3 regions, spanning from 3 rd March to 12th May (61 days). The dataset is structured as a collection of graph snapshots, where each snapshot represents a specific day. Within each graph, the nodes correspond to the 129 regions in England. The node features capture the number of COVID-19 cases reported in each region over the past 8 days, while the objective is to predict the number of cases in each node for the following day. The graphs are directed and weighted, with edge weights indicating the daily volume of people moving from one region to another. These weights are derived from the Facebook Data For Good disease prevention maps and the official U.K. government website. Importantly, the graph snapshots are dynamic, meaning that different snapshots exhibit variations in terms of node features, edge weights, and prediction targets. To ensure consistent analysis, we normalize the reported cases for both node features and targets, setting their mean to zero and their variance to one. We divide the snapshots into training, validation, and test sets using a 2:2:6 ratio chronologically.

To address the challenge of missing data, we apply a masking strategy when constructing the dataset, which randomly hides a portion of the reported cases with a probability of 0.9. Notice that a reported case can be used both as part of node feature and as target. When missing data occurs in the node features, we substitute it with a value of zero. As for the missing data in the target, we treat it as NaN (Not a Number). This means that on the specific day (graph snapshot), the region with missing target data does not contribute to the neural network training. We achieve this by masking out the loss when the target is NaN during training. This setting effectively simulates the scenario of missing data that occurs in real-world situations, closely resembling the challenges encountered in practical applications.

TABLE II
AVERAGE ACCURACY (IN %) AND 95% CONFIDENCE INTERVAL IN *mini*IMAGENET, *tiered*IMAGENET AND CUB

Methods	Backbone	*mini*ImageNet		*tiered*ImageNet		CUB	
		1-shot	5-shot	1-shot	5-shot	1-shot	5-shot
MAML [65]	ResNet-18	49.61 ± 0.92	65.72 ± 0.77	-	-	69.96 ± 1.01	82.70 ± 0.65
Baseline [66]	ResNet-18	51.87 ± 0.77	75.68 ± 0.63	-	-	67.02 ± 0.90	83.58 ± 0.54
RelationNet [67]	ResNet-18	52.48 ± 0.86	69.83 ± 0.68	54.48 ± 0.93	71.32 ± 0.78	67.59 ± 1.02	82.75 ± 0.58
MatchingNet [63]	ResNet-18	52.91 ± 0.88	68.88 ± 0.69	-	-	72.36 ± 0.90	83.64 ± 0.60
ProtoNet [68]	ResNet-18	54.16 ± 0.82	73.68 ± 0.65	53.31 ± 0.89	72.69 ± 0.74	71.88 ± 0.91	87.42 ± 0.48
Gidaris [69]	ResNet-15	55.45 ± 0.89	70.13 ± 0.68	-	-	-	-
SNAIL [70]	ResNet-15	55.71 ± 0.99	68.88 ± 0.92	-	-	-	-
TADAM [71]	ResNet-15	58.50 ± 0.30	76.70 ± 0.30	-	-	-	-
Transductive [72]	ResNet-12	62.35 ± 0.66	74.53 ± 0.54	-	-	-	-
MetaoptNet [73]	ResNet-18	62.64 ± 0.61	78.63 ± 0.46	65.99 ± 0.72	81.56 ± 0.53	-	-
TPN [74]	ResNet-12	53.75 ± 0.86	69.43 ± 0.67	57.53 ± 0.96	72.85 ± 0.74	-	-
TEAM [75]	ResNet-18	60.07 ± 0.59	75.90 ± 0.38	-	-	80.16 ± 0.52	87.17 ± 0.39
CAN+T [76]	ResNet-12	67.19 ± 0.55	80.64 ± 0.35	73.21 ± 0.58	84.93 ± 0.38	-	-
SimpleShot [77]†	ResNet-18	62.86 ± 0.20	79.22 ± 0.14	69.71 ± 0.23	84.13 ± 0.17	72.86 ± 0.20	88.57 ± 0.11
LaplacianShot [78]†	ResNet-18	70.46 ± 0.23	81.76 ± 0.14	76.90 ± 0.25	85.10 ± 0.17	82.92 ± 0.21	90.77 ± 0.11
EPNet [79]†	ResNet-18	63.83 ± 0.20	77.98 ± 0.15	70.08 ± 0.23	82.11 ± 0.18	73.32 ± 0.21	87.55 ± 0.13
Diff-ResNet [38]†	ResNet-18	71.11 ± 0.24	82.07 ± 0.14	77.98 ± 0.25	85.75 ± 0.17	84.20 ± 0.21	91.12 ± 0.10
COIN	ResNet-18	**72.50** ± 0.24	**82.07** ± 0.14	**78.83** ± 0.25	**86.05** ± 0.16	**85.63** ± 0.20	**91.31** ± 0.10
Qiao [80]	WRN	59.60 ± 0.41	73.74 ± 0.19	-	-	-	-
LEO [81]	WRN	61.76 ± 0.08	77.59 ± 0.12	66.33 ± 0.05	81.44 ± 0.09	-	-
ProtoNet [68]	WRN	62.60 ± 0.20	79.97 ± 0.14	-	-	-	-
CC+rot [82]	WRN	62.93 ± 0.45	79.87 ± 0.33	70.53 ± 0.51	84.98 ± 0.36	-	-
MatchingNet [63]	WRN	64.03 ± 0.20	76.32 ± 0.16	-	-	-	-
FEAT [83]	WRN	65.10 ± 0.20	81.11 ± 0.14	70.41 ± 0.23	84.38 ± 0.16	-	-
Transductive [72]	WRN	65.73 ± 0.68	78.40 ± 0.52	73.34 ± 0.71	85.50 ± 0.50	-	-
BD-CSPN [84]	WRN	70.31 ± 0.93	81.89 ± 0.60	78.74 ± 0.95	86.92 ± 0.63	-	-
PT+NCM [85]	WRN	65.35 ± 0.20	**83.87** ± 0.13	69.96 ± 0.22	86.45 ± 0.15	80.57 ± 0.20	91.15 ± 0.10
SimpleShot [77]†	WRN	65.20 ± 0.20	81.28 ± 0.14	71.49 ± 0.23	85.51 ± 0.16	78.62 ± 0.19	91.21 ± 0.10
LaplacianShot [78]†	WRN	72.90 ± 0.23	83.47 ± 0.14	78.79 ± 0.25	86.46 ± 0.17	87.70 ± 0.18	92.73 ± 0.10
EPNet [79]†	WRN	67.09 ± 0.21	80.71 ± 0.14	73.20 ± 0.23	84.20 ± 0.17	80.88 ± 0.20	91.40 ± 0.11
Diff-ResNet [38]†	WRN	73.47 ± 0.23	83.86 ± 0.14	79.74 ± 0.25	87.10 ± 0.16	87.74 ± 0.19	92.96 ± 0.09
COIN	WRN	**74.85** ± 0.24	83.82 ± 0.14	**80.66** ± 0.25	**87.48** ± 0.15	**89.20** ± 0.18	**93.24** ± 0.09

Re-implemented results using public official code with our pretrained backbone are marked with †.

However, during testing, we make an exception and use the true target values for more accurate evaluation. Nevertheless, any missing data in the node feature is still treated as zero.

We compared our COIN model with several methods using Mean Squared Error (MSE) on the test set as the evaluation metric. To provide a fair benchmark, we establish a baseline where we simply predict all-zero values. Since the dataset has been normalized to zero mean, this baseline serves as a reference point. Additionally, we implemented Logistic Regression (LR), Multi-Layer Perceptron (MLP), and Graph Convolutional Network (GCN) to compare against our COIN model. These models represent traditional and widely-used approaches in the field. Furthermore, we explore various methods that leverage techniques from recurrent neural networks (RNN) to incorporate temporal information. These methods not only consider spatial information but also incorporate temporal information.

We randomly mask out the reported cases using 10 random seeds, each with 10 random neural network initialization. The results are presented in Table III. In our experimental setup, where a significant portion of the data is missing, methods that incorporate temporal information struggle to generalize and some perform even worse than the all-zero baseline. We speculate that the misleading effect of missing data on network predictions is amplified when employing temporal information within an RNN structure.

TABLE III
AVERAGE MEAN SQUARED ERROR ON ENGLAND COVID-19 DATASET WITH MISSING DATA

Method	MSE
All-Zero	0.8197
LR	0.8344 ± 0.0769
MLP	0.7777 ± 0.0203
GCN	0.7607 ± 0.0185
DCRNN [96]	0.8213 ± 0.0663
MPNN-LSTM [92]	0.9384 ± 0.0633
GConvGRU [97]	0.8153 ± 0.0435
A3TGCN [98]	0.7797 ± 0.0362
EvolveGCN [99]	0.8237 ± 0.0856
COIN	**0.7168** ± 0.0187

Conversely, non-RNN methods such as MLP and GCN outperform the aforementioned approaches. The superiority of GCN over MLP can be attributed to the inclusion of graph information, specifically the daily movement of people, which plays a crucial role in pandemic prediction, especially when only a limited amount of valid data from the past several days is available in our setting. By leveraging information from neighboring nodes, the neural network can make more accurate predictions.

Nevertheless, our COIN model surpasses all other methods by a significant margin, with a notable 74.4% improvement over GCN when compared to the all-zero baseline.

TABLE IV
AVERAGE ACCURACY, ROC-AUC AND AUPRC ON PROSTATE CLASSIFICATION TASK

Method	Accuracy(%)	ROC-AUC	AUPRC
LR	67.90	0.5884	0.4498
SVM	67.01	0.6484	0.4370
Random forest	67.14	0.7307	0.5150
AdaBoost	76.75	0.7094	0.5967
MLP	73.39	0.6540	0.5498
P-NET [102]	74.77	0.7720	0.6548
ResNet	78.23	0.7635	0.7100
COIN	**80.35**	**0.8001**	**0.7598**

D. Prostate Cancer Classification

Over the past decade, significant advancements in molecular profiling technologies have enabled the collection of genomic, transcriptional and additional features from cancer patients. This wealth of molecular profiling data, combined with clinical annotations, has greatly contributed to the identification of numerous genes, pathways, and complexes associated with lethal cancers. However, in the case of prostate cancer, uncovering the relationship between these molecular features and disease prediction remains a major biological and clinical challenge [100], [101], [102]. Developing a highly accurate predictive model is essential for the early detection of preclinical prostate cancer and holds significant potential for wider application in various cancer types.

The dataset utilized in this study is derived from the work of Armenia et al. [103], where they collected and analyzed genomic profiles from 1,013 patients diagnosed with prostate cancer (680 primary and 333 metastatic). These profiles are generated using a unified computational pipeline to ensure consistent derivation of somatic alterations. Following [102], patient features are aggregated at the gene level, resulting in a total of 9,229 genes. Each gene is encoded with binary values (0 or 1) to represent somatic mutation, copy number amplification, and copy number deletion. Consequently, each patient is characterized by a feature vector of dimension 27,687. The dataset is split into a training set of size 20, a validation set of size 100 and the rest as the test set. The objective is to predict the cancer state of patients in the test set as either primary or metastatic.

We compare our COIN model with several machine learning methods and a deep neural network approach. Machine learning methods include logistic regression (LR), support vector machine (SVM), random forest and adaptive boosting (AdaBoost). The deep neural network approach, P-NET [102], incorporates curated biological pathways to build a pathway-aware multi-layered hierarchical network. In this network, each neuron represents a gene, and the connections between neurons correspond to biological pathways. As a result, P-NET is a sparse network with 6 layers and a total of 71,009 parameters.

Following [102], we adopt the same first layer structure from [102] in Multi-Layer Perceptron (MLP), ResNet and COIN in Table IV. Specifically, each neuron is connected to exactly three nodes in the input feature representing somatic mutation, copy number amplification, and copy number deletion of one gene. To ensure that the performance improvement of our COIN over P-NET does not result from an increase in parameters, the hidden dimension of MLP, ResNet and COIN is set to 3. Consequently, the total number of parameters is 64610 for MLP, and 73840 for ResNet and COIN, which is comparable to that of the P-NET model. Similar to few-shot learning, the weight is computed using a Gaussian kernel based on the Euclidean distance between patient features. However, instead of using the raw 27,687 dimension feature, we pretrain a ResNet and select the 9,229 dimension output after the first layer as the feature for computing distance.

We conduct a comparison of the accuracy, area under the receiver operating characteristic curve (ROC-AUC), and area under the precision-recall curve (AUPRC) for these methods. ROC and PRC shows the true positive rate v.s. false positive rate, and precision v.s. recall, respectively, at different classification thresholds. The results, presented in Table IV, are averaged over 10 random train-validation-test splits and 10 random initializations for each split. Notably, our COIN consistently achieves the highest performance across all evaluation metrics. Compared with ResNet, out COIN adds several diffusion layers motivated by PDE framework and further improves the classification performance. These results underscore the effectiveness of our axiomatic framework in accurately classifying prostate cancer state.

V. CONCLUSION

Motivated by the scale-space theory, we theoretically prove that the evolution from a base classifier to neural networks can be modeled by a convection-diffusion equation. Based on the theoretical results, we develop a novel network structure and verify its effectiveness through extensive experiments. We are aware that modeling the convection-diffusion equation through introducing diffusion mechanism is one of the many possible approaches, and we are looking forward to explore other paths in the future work.

REFERENCES

[1] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–14.
[2] G. E. Dahl, D. Yu, L. Deng, and A. Acero, "Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 20, no. 1, pp. 30–42, Jan. 2012.
[3] L. Bo, K. Lai, X. Ren, and D. Fox, "Object recognition with hierarchical kernel descriptors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2011, pp. 1729–1736.
[4] L. Wang et al., "Temporal segment networks: Towards good practices for deep action recognition," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 20–36.
[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[6] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 630–645.
[7] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proc. Brit. Mach. Vis. Conf.*, 2016, pp. 87.1–87.12.
[8] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1492–1500.

[9] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis/ Pattern Recognit.*, 2017, pp. 4700–4708.

[10] E. Weinan, "A proposal on machine learning via dynamical systems," *Commun. Math. Statist.*, vol. 5, no. 1, pp. 1–11, 2017.

[11] E. Haber, L. Ruthotto, E. Holtham, and S.-H. Jun, "Learning across scales—Multiscale methods for convolution neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3142–3148.

[12] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural ordinary differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6572–6583.

[13] G. Larsson, M. Maire, and G. Shakhnarovich, "Fractalnet: Ultra-deep neural networks without residuals," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–11.

[14] X. Zhang, Z. Li, C. Change Loy, and D. Lin, "Polynet: A pursuit of structural diversity in very deep networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 718–726.

[15] Y. Lu, A. Zhong, Q. Li, and B. Dong, "Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 3276–3285.

[16] J. Jia and A. R. Benson, "Neural jump stochastic differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 9843–9854.

[17] S. Sonoda and N. Murata, "Transport analysis of infinitely deep neural network," *J. Mach. Learn. Res.*, vol. 20, no. 1, pp. 31–82, 2019.

[18] T. Y. S. Qi and D. Qiang, "Stochastic training of residual networks in deep learning," *Mathematica Numerica Sinica*, vol. 42, no. 3, pp. 349–369, 2020.

[19] B. Wang, B. Yuan, Z. Shi, and S. J. Osher, "EnResNet: ResNets ensemble via the Feynman–Kac formalism for adversarial defense and beyond," *SIAM J. Math. Data Sci.*, vol. 2, no. 3, pp. 559–582, 2020.

[20] J. J. Koenderink, "The structure of images," *Biol. Cybern.*, vol. 50, no. 5, pp. 363–370, 1984.

[21] A. P. Witkin, "Scale-space filtering," in *Readings in Computer Vision*. Amsterdam, Netherlands: Elsevier, 1987, pp. 329–332.

[22] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 8, no. 6, pp. 679–698, Nov. 1986.

[23] P. Perona and J. Malik, "Scale-space and edge detection using anisotropic diffusion," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 12, no. 7, pp. 629–639, Jul. 1990.

[24] L. Alvarez, F. Guichard, P.-L. Lions, and J.-M. Morel, "Axioms and fundamental equations of image processing," *Arch. Rational Mechanics Anal.*, vol. 123, no. 3, pp. 199–257, 1993.

[25] R. Duits, L. Florack, J. De Graaf, and B. T. HaarRomeny, "On the axioms of scale space theory," *J. Math. Imag. Vis.*, vol. 20, pp. 267–298, 2004.

[26] W. Tao, H. Ling, Z. Shi, and B. Wang, "Deep learning with data privacy via residual perturbation," 2024, *arXiv:2408.05723*.

[27] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[28] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 84–90.

[29] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[30] X. Liu, S. Si, Q. Cao, S. Kumar, and C.-J. Hsieh, "How does noise help robustness? Explanation and exploration under the neural sde framework," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 282–290.

[31] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[32] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1310–1320.

[33] B. Li, C. Chen, W. Wang, and L. Carin, "Certified adversarial robustness with additive noise," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 9464–9474.

[34] H. Salman et al., "Provably robust deep learning via adversarially trained smoothed classifiers," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 11292–11303.

[35] M. Eliasof, E. Haber, and E. Treister, "PDE-GCN: Novel architectures for graph neural networks motivated by partial differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, Art. no. 293.

[36] B. Chamberlain, J. Rowbottom, M. I. Gorinova, M. Bronstein, S. Webb, and E. Rossi, "Grand: Graph neural diffusion," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1407–1418.

[37] Y. Wang, Y. Wang, J. Yang, and Z. Lin, "Dissecting the diffusion process in linear graph convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5758–5769.

[38] T. Wang, Z. Dou, C. Bao, and Z. Shi, "Diffusion mechanism in residual neural network: Theory and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 2, pp. 667–680, Feb. 2024.

[39] L.-P. Xhonneux, M. Qu, and J. Tang, "Continuous graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10432–10441.

[40] M. Poli, S. Massaroli, J. Park, A. Yamashita, H. Asama, and J. Park, "Graph neural ordinary differential equations," 2021, *arXiv:1911.07532*.

[41] X. Mao, *Stochastic Differential Equations and Applications*. Amsterdam, Netherlands: Elsevier, 2007.

[42] X. Gastaldi, "Shake-shake regularization," 2017, *arXiv:1705.07485*.

[43] C.-W. Huang and S. S. Narayanan, "Stochastic Shake-Shake regularization for affective learning from speech," in *Proc. INTERSPEECH*, 2018, pp. 3658–3662.

[44] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep networks with stochastic depth," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 646–661.

[45] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. Boca Raton, FL, USA: CRC Press, 2012.

[46] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–11.

[47] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Proc. Int. Conf. Learn. Representations, Workshop Track*, 2017, pp. 1–14.

[48] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[49] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.

[50] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–93, 2008.

[51] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," 2019.

[52] Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 40–48.

[53] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.

[54] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.

[55] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.

[56] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.

[57] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–15.

[58] J. Gasteiger, S. Weißenberger, and S. Günnemann, "Diffusion improves graph learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 13366–13378.

[59] B. Xu, H. Shen, Q. Cao, K. Cen, and X. Cheng, "Graph convolutional networks using heat kernel for semi-supervised learning," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 1928–1934.

[60] Q. Wu, C. Yang, W. Zhao, Y. He, D. Wipf, and J. Yan, "Difformer: Scalable (graph) transformers induced by energy constrained diffusion," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–26.

[61] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," 2018, *arXiv: 1802.03426*.

[62] L. Fei-Fei, R. Fergus, and P. Perona, "One-shot learning of object categories," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 4, pp. 594–611, Apr. 2006.

[63] O. Vinyals et al., "Matching networks for one shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3630–3638.
[64] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM Comput. Surv.*, vol. 53, no. 3, pp. 1–34, 2020.
[65] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
[66] W.-Y. Chen, Y.-C. Liu, Z. Kira, Y.-C. Wang, and J.-B. Huang, "A closer look at few-shot classification," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–16.
[67] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1199–1208.
[68] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4077–4087.
[69] S. Gidaris and N. Komodakis, "Dynamic few-shot visual learning without forgetting," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4367–4375.
[70] N. Mishra, M. Rohaninejad, X. Chen, and P. Abbeel, "A simple neural attentive meta-learner," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–17.
[71] B. Oreshkin, P. Rodr íguez López, and A. Lacoste, "Tadam: Task dependent adaptive metric for improved few-shot learning," in *Proc Adv. Neural Inf. Process. Syst.*, 2018, pp. 719–729.
[72] G. S. Dhillon, P. Chaudhari, A. Ravichandran, and S. Soatto, "A baseline for few-shot image classification," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–20.
[73] K. Lee, S. Maji, A. Ravichandran, and S. Soatto, "Meta-learning with differentiable convex optimization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10657–10665.
[74] Y. Liu et al., "Learning to propagate labels: Transductive propagation network for few-shot learning," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–14.
[75] L. Qiao, Y. Shi, J. Li, Y. Wang, T. Huang, and Y. Tian, "Transductive episodic-wise adaptive metric for few-shot learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3603–3612.
[76] R. Hou, H. Chang, B. Ma, S. Shan, and X. Chen, "Cross attention network for few-shot classification," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4005–4016.
[77] Y. Wang, W.-L. Chao, K. Q. Weinberger, and L. van der Maaten, "Simpleshot: Revisiting nearest-neighbor classification for few-shot learning," 2019, *arXiv:1911.04623*.
[78] I. Ziko, J. Dolz, E. Granger, and I. B. Ayed, "Laplacian regularized few-shot learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 11660–11670.
[79] P. Rodríguez, I. Laradji, A. Drouin, and A. Lacoste, "Embedding propagation: Smoother manifold for few-shot classification," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 121–138.
[80] S. Qiao, C. Liu, W. Shen, and A. L. Yuille, "Few-shot image recognition by predicting parameters from activations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7229–7238.
[81] A. A. Rusu et al., "Meta-learning with latent embedding optimization," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–17.
[82] S. Gidaris, A. Bursuc, N. Komodakis, P. Pérez, and M. Cord, "Boosting few-shot visual learning with self-supervision," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 8059–8068.
[83] H.-J. Ye, H. Hu, D.-C. Zhan, and F. Sha, "Few-shot learning via embedding adaptation with set-to-set functions," in *Proc. IEEE/CVF Conf. Comput. Vis. pattern Recognit.*, 2020, pp. 8808–8817.
[84] J. Liu, L. Song, and Y. Qin, "Prototype rectification for few-shot learning," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 741–756.
[85] Y. Hu, V. Gripon, and S. Pateux, "Leveraging the feature distribution in transfer-based few-shot learning," in *Proc. Int. Conf. Artif. Neural Netw.*, 2021, pp. 487–499.
[86] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "*The Caltech-UCSD Birds-200–2011 Dataset*," California Inst. Technol., Pasadena, CA, USA, Rep. CNS-TR-2011-001, 2011.
[87] A. Zeroual, F. Harrou, A. Dairi, and Y. Sun, "Deep learning methods for forecasting covid-19 time-series data: A comparative study," *Chaos Solitons Fractals*, vol. 140, 2020, Art. no. 110121.
[88] V. K. R. Chimmula and L. Zhang, "Time series forecasting of COVID-19 transmission in Canada using LSTM networks," *Chaos Solitons Fractals*, vol. 135, 2020, Art. no. 109864.
[89] R. Pal, A. A. Sekh, S. Kar, and D. K. Prasad, "Neural network based country wise risk prediction of COVID-19," *Appl. Sci.*, vol. 10, no. 18, 2020, Art. no. 6448.
[90] Z. Hu, Q. Ge, S. Li, L. Jin, and M. Xiong, "Artificial intelligence forecasting of COVID-19 in China," 2020, *arXiv:2002.07112*.
[91] A. Kapoor et al., "Examining COVID-19 forecasting using spatio-temporal graph neural networks," 2020, *arXiv:2007.03113*.
[92] G. Panagopoulos, G. Nikolentzos, and M. Vazirgiannis, "Transfer graph neural networks for pandemic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4838–4845.
[93] J. Gao et al., "STAN: Spatio-temporal attention network for pandemic prediction using real-world evidence," *J. Amer. Med. Informat. Assoc.*, vol. 28, no. 4, pp. 733–743, 2021.
[94] C. Fritz, E. Dorigatti, and D. Rügamer, "Combining graph neural networks and spatio-temporal disease models to improve the prediction of weekly COVID-19 cases in Germany," *Sci. Rep.*, vol. 12, no. 1, 2022, Art. no. 3930.
[95] B. Rozemberczki et al., "Pytorch geometric temporal: Spatiotemporal signal processing with neural machine learning models," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 4564–4573.
[96] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–16.
[97] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, "Structured sequence modeling with graph convolutional recurrent networks," in *Proc. 25th Int. Conf. Neural Inf. Process.*, Siem Reap, Cambodia, 2018, pp. 362–373.
[98] J. Bai et al., "A3T-GCN: Attention temporal graph convolutional network for traffic forecasting," *ISPRS Int. J. Geo- Inf.*, vol. 10, no. 7, 2021, Art. no. 485.
[99] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 5363–5370.
[100] D. Robinson et al., "Integrative clinical genomics of advanced prostate cancer," *Cell*, vol. 161, no. 5, pp. 1215–1228, 2015.
[101] W. Abida et al., "Genomic correlates of clinical outcome in advanced prostate cancer," *Proc. Nat. Acad. Sci. USA*, vol. 116, no. 23, pp. 11428–11436, 2019.
[102] H. A. Elmarakeby et al., "Biologically informed deep neural network for prostate cancer discovery," *Nature*, vol. 598, no. 7880, pp. 348–352, 2021.
[103] J. Armenia et al., "The long tail of oncogenic drivers in prostate cancer," *Nature Genet.*, vol. 50, no. 5, pp. 645–651, 2018.
[104] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUS)," 2016, *arXiv:1511.07289*.

Tangjun Wang is currently working toward the PhD degree with the Department of Mathematical Sciences, Tsinghua University supervised by Zuoqiang Shi. His research interests include semi-supervised learning, and applications of partial differential equation in neural networks.

Chenglong Bao received the PhD degree from the Department of Mathematics, National University of Singapore, in 2014. He is an assistant professor with Yau Mathematical Sciences Center, Tsinghua University and Yanqi Lake Beijing Institute of Mathematical Sciences and Applications. His main research interests include mathematical image processing, large scale optimization, and its applications.

Zuoqiang Shi received the PhD degree from Zhou Pei-Yuan center for Applied Mathematics, Tsinghua University, in 2008. He is a professor with Yau Mathematical Sciences Center, Tsinghua University. His main research interests include numerical methods of partial differential equations and its applications and mathematical image processing.